

1. Preguntas sobre el ejemplo de clasificación de imágenes con PyTorch y MLP

1. 1. Dataset y Preprocesamiento

- 1. 1. 1. ¿Por qué es necesario redimensionar las imágenes a un tamaño fijo para una MLP?
- 1. 1. 2. ¿Qué ventajas ofrece Albumentations frente a otras librerías de transformación como `torchvision.transforms`?
- 1. 1. 3. ¿Qué hace `A.Normalize()`? ¿Por qué es importante antes de entrenar una red?
- 1. 1. 4. ¿Por qué convertimos las imágenes a `ToTensorV2()` al final de la pipeline?

1. 2. 2. Arquitectura del Modelo

- 1. 2. 1. ¿Por qué usamos una red MLP en lugar de una CNN aquí? ¿Qué limitaciones tiene?
- 1. 2. 2. ¿Qué hace la capa `Flatten()` al principio de la red?
- 1. 2. 3. ¿Qué función de activación se usó? ¿Por qué no usamos Sigmoid o Tanh?
- 1. 2. 4. ¿Qué parámetro del modelo deberíamos cambiar si aumentamos el tamaño de entrada de la imagen?

1. 3. 3. Entrenamiento y Optimización

- 1. 3. 1. ¿Qué hace `optimizer.zero_grad()`?
- 1. 3. 2. ¿Por qué usamos `CrossEntropyLoss()` en este caso?
- 1. 3. 3. ¿Cómo afecta la elección del tamaño de batch (`batch_size`) al entrenamiento?
- 1. 3. 4. ¿Qué pasaría si no usamos `model.eval()` durante la validación?

1. 4. 4. Validación y Evaluación

- 1. 4. 1. ¿Qué significa una accuracy del 70% en validación pero 90% en entrenamiento?
- 1. 4. 2. ¿Qué otras métricas podrían ser más relevantes que accuracy en un problema real?
- 1. 4. 3. ¿Qué información útil nos da una matriz de confusión que no nos da la accuracy?
- 1. 4. 4. En el reporte de clasificación, ¿qué representan precision, recall y f1-score?

1. 5. 5. TensorBoard y Logging

- 1. 5. 1. ¿Qué ventajas tiene usar TensorBoard durante el entrenamiento?
- 1. 5. 2. ¿Qué diferencias hay entre logear `add_scalar`, `add_image` y `add_text`?
- 1. 5. 3. ¿Por qué es útil guardar visualmente las imágenes de validación en TensorBoard?
- 1. 5. 4. ¿Cómo se puede comparar el desempeño de distintos experimentos en TensorBoard?

1. 6. 6. Generalización y Transferencia

- 1. 6. 1. ¿Qué cambios habría que hacer si quisiéramos aplicar este mismo modelo a un dataset con 100 clases?
- 1. 6. 2. ¿Por qué una CNN suele ser más adecuada que una MLP para clasificación de imágenes?
- 1. 6. 3. ¿Qué problema podríamos tener si entrenamos este modelo con muy pocas imágenes por clase?

1. 6. 4. ¿Cómo podríamos adaptar este pipeline para imágenes en escala de grises?

2. Regularización

2. 1. Preguntas teóricas:

2. 1. 1. ¿Qué es la regularización en el contexto del entrenamiento de redes neuronales?

2. 1. 2. ¿Cuál es la diferencia entre Dropout y regularización L2 (weight decay)?

2. 1. 3. ¿Qué es BatchNorm y cómo ayuda a estabilizar el entrenamiento?

2. 1. 4. ¿Cómo se relaciona BatchNorm con la velocidad de convergencia?

2. 1. 5. ¿Puede BatchNorm actuar como regularizador? ¿Por qué?

2. 1. 6. ¿Qué efectos visuales podrías observar en TensorBoard si hay overfitting?

2. 1. 7. ¿Cómo ayuda la regularización a mejorar la generalización del modelo?

2. 2. Actividades de modificación:

2. 2. 1. Agregar Dropout en la arquitectura MLP

- Insertar capas `nn.Dropout(p=0.5)` entre las capas lineales y activaciones.
- Comparar los resultados con y sin Dropout.

2. 2. 2. Agregar Batch Normalization

Insertar `nn.BatchNorm1d(...)` después de cada capa Linear y antes de la activación:

```
1 self.net = nn.Sequential(  
2     nn.Flatten(),  
3     nn.Linear(in_features, 512),  
4     nn.BatchNorm1d(512),  
5     nn.ReLU(),  
6     nn.Dropout(0.5),  
7     nn.Linear(512, 256),  
8     nn.BatchNorm1d(256),  
9     nn.ReLU(),  
10    nn.Dropout(0.5),  
11    nn.Linear(256, num_classes)  
12 )
```

 Python

2. 2. 3. Aplicar Weight Decay (L2)

Modificar el optimizador:

```
1 optimizer = torch.optim.Adam(model.parameters(), lr=0.001,  
2 weight_decay=1e-4)
```

 Python

2. 2. 4. Reducir overfitting con data augmentation

Agregar transformaciones en Albumentations como `HorizontalFlip`, `BrightnessContrast`, `ShiftScaleRotate`.

2. 2. 5. Early Stopping (opcional)

Implementar un criterio para detener el entrenamiento si la validación no mejora después de N épocas.

2. 3. Preguntas prácticas:

- 2. 3. 1. ¿Qué efecto tuvo BatchNorm en la estabilidad y velocidad del entrenamiento?
- 2. 3. 2. ¿Cambió la performance de validación al combinar BatchNorm con Dropout?
- 2. 3. 3. ¿Qué combinación de regularizadores dio mejores resultados en tus pruebas?
- 2. 3. 4. ¿Notaste cambios en la loss de entrenamiento al usar BatchNorm?

3. Inicialización de Parámetros

3. 1. Preguntas teóricas:

- 3. 1. 1. ¿Por qué es importante la inicialización de los pesos en una red neuronal?
- 3. 1. 2. ¿Qué podría ocurrir si todos los pesos se inicializan con el mismo valor?
- 3. 1. 3. ¿Cuál es la diferencia entre las inicializaciones de Xavier (Glorot) y He?
- 3. 1. 4. ¿Por qué en una red con ReLU suele usarse la inicialización de He?
- 3. 1. 5. ¿Qué capas de una red requieren inicialización explícita y cuáles no?

3. 2. Actividades de modificación:

3. 2. 1. Agregar inicialización manual en el modelo

En la clase MLP, agregar un método `init_weights` que inicialice cada capa:

```
1 def init_weights(self):
2     for m in self.modules():
3         if isinstance(m, nn.Linear):
4             nn.init.kaiming_normal_(m.weight)
5             nn.init.zeros_(m.bias)
```

 Python

3. 2. 2. Probar distintas estrategias de inicialización

```
1 - Xavier (`nn.init.xavier_uniform_`)
2 - He (`nn.init.kaiming_normal_`)
3 - Aleatoria uniforme (`nn.init.uniform_`)
4 - Comparar la estabilidad y velocidad del entrenamiento.
```

 Python

3. 2. 3. Visualizar pesos en TensorBoard:

Agregar esta línea en la primera época para observar los histogramas:

```
1 for name, param in model.named_parameters():
2     writer.add_histogram(name, param, epoch)
```

 Python

3. 3. Preguntas prácticas:

3. 3. 1. ¿Qué diferencias notaste en la convergencia del modelo según la inicialización?

3. 3. 2. ¿Alguna inicialización provocó inestabilidad (pérdida muy alta o NaNs)?

3. 3. 3. ¿Qué impacto tiene la inicialización sobre las métricas de validación?

3. 3. 4. ¿Por qué bias se suele inicializar en cero?